

### 3. Data Link Layer

Indian Institute of Information Technology Guwahati

August 19, 2026

# Objectives

- Understand the design principles of DLL algorithms for achieving reliable, efficient communication of whole units of information called frames between two **adjacent machines**.
- Framing, Error correction and Flow Control.

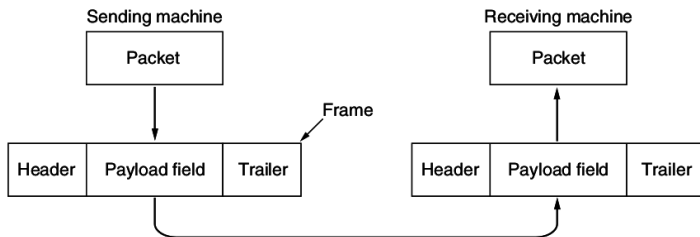
The data link layer uses the services of the physical layer to send and receive bits over communication channels. It has a number of functions, including:

- Providing a well-defined service interface to the network layer.
- Dealing with transmission errors.
- Regulating the flow of data so that slow receivers are not swamped by fast senders.

# DLL Design Issues

The data link layer takes the packets it gets from the network layer and encapsulates them into frames for transmission.

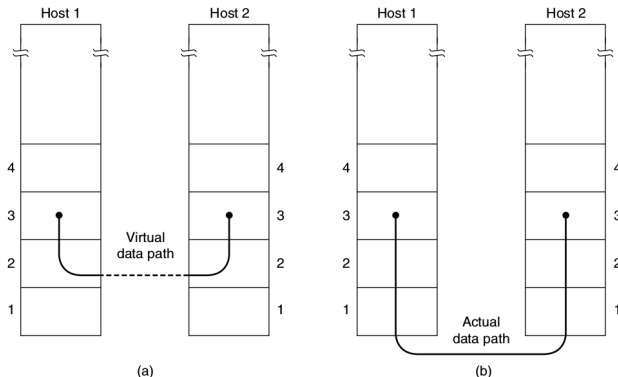
Each frame contains a frame header, a payload field for holding the packet, and a frame trailer.



**Figure 3-1.** Relationship between packets and frames.

# Services Provided to the Network Layer

The principal service is transferring data from the network layer on the source machine to the network layer on the destination machine.



**Figure 3-2.** (a) Virtual communication. (b) Actual communication.

# Types of Services

## Unacknowledged connectionless service

- Frame is sent with no connection / error recovery
- Ethernet is example

## Acknowledged connectionless service

- Frame is sent with retransmissions if needed
- Example is 802.11

## Acknowledged connection-oriented

- connection oriented service
- Connection is set up; rare

To provide service to the network layer, the data link layer must use the service provided to it by the physical layer.

The physical layer simply accepts a raw bit stream and attempt to deliver it to the destination. If the channel is noisy, the physical layer will add some redundancy to its signals to reduce the bit error rate to a tolerable level.

However, the bit stream received by the data link layer is not guaranteed to be error free.

**It is up to the data link layer to detect and, if necessary, correct errors.**

# Framing

The data link layer breaks up the bit stream into discrete frames, compute a short token called a checksum for each frame, and include the checksum in the frame when it is transmitted.

Breaking up the bit stream into frames is not straight forward.

A good design must **make it easy for a receiver to find the start of new frames** while using little of the channel bandwidth.

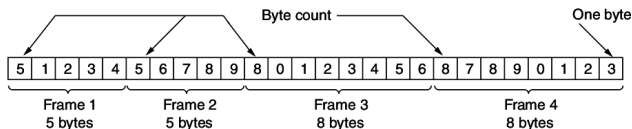


# Framing: 4 Methods

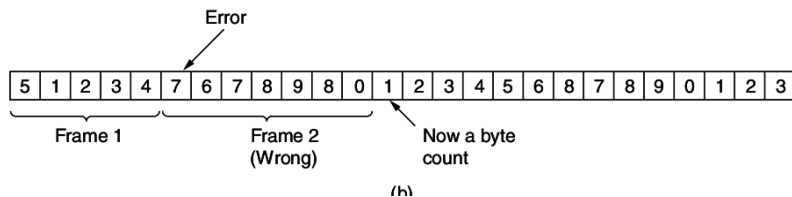
- ① Byte count.
- ② Flag bytes with byte stuffing.
- ③ Flag bits with bit stuffing.
- ④ Physical layer coding violations.

# Byte count

- This framing method uses a field in the header to specify the number of bytes in the frame.
- When the data link layer at the destination sees the byte count, it knows how many bytes follow and hence where the end of the frame is.



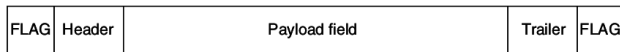
# Byte count



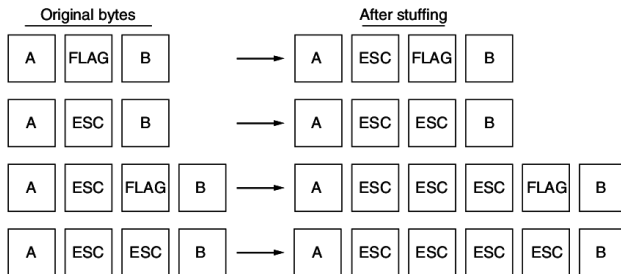
# Flag bytes with byte stuffing

Resynchronization after an error is addressed by having each frame start and end with special bytes.

- **Flag byte**, is used as both the starting and ending delimiter.
- Two consecutive flag bytes indicate the end of one frame and the start of the next.
- If the receiver ever loses synchronization it can just search for two flag bytes to find the end of the current frame and the start of the next frame.



# Flag bytes with byte stuffing



PPP (Point-to-Point Protocol) is the standard example of byte stuffing, using 0x7E as the flag byte and 0x7D as the escape byte.

# Flag bytes with bit stuffing

- Each frame begins and ends with a special bit pattern, **01111110** or **0x7E** in hexadecimal.
- Whenever the sender's data link layer encounters five consecutive 1s in the data, it automatically stuffs a 0 bit into the outgoing bit stream.
- It also ensures a minimum density of transitions that help the physical layer maintain synchronization.

```
0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0  
  
0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0  
                ↙       ↑       ↘  
                Stuffed bits  
  
0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0
```

# Flag bytes with bit stuffing

- The length of a frame now depends on the contents of the data it carries
- If there are no flag bytes in the data, 100 bytes might be carried in a frame of roughly 100 bytes.
- If the data consists solely of **flag bytes** the frame will become roughly 200 bytes long.
- With bit stuffing, the increase would be roughly 12.5% as 1 bit is added to every byte

HDLC (High-Level Data Link Control) is the standard example of bit stuffing, using the flag 01111110 (0x7E) as the frame delimiter.

# Physical layer coding violations

In 4B/5B line code 4 data bits are mapped to 5 signal bits to ensure sufficient bit transitions.

|   | 4-bit | 5-bit |
|---|-------|-------|
| 0 | 0000  | 11110 |
| 1 | 0001  | 01001 |
| 2 | 0010  | 10100 |
| 3 | 0011  | 10101 |
| 4 | 0100  | 01010 |
| 5 | 0101  | 01011 |
| 6 | 0110  | 01110 |
| 7 | 0111  | 01111 |
| 8 | 1000  | 10010 |
| 9 | 1001  | 10011 |
| A | 1010  | 10110 |
| B | 1011  | 10111 |
| C | 1100  | 11010 |
| D | 1101  | 11011 |
| E | 1110  | 11100 |
| F | 1111  | 11101 |



# Physical layer coding violations contd.

The 5-bit codes are chosen so that:

- Each code has at least two 1s.
- No code has more than one leading 0 or more than two trailing 0s.
- When you concatenate codes, you get no more than 3 consecutive 0s overall.

# Physical layer coding violations contd.

Redundancy in encoding of bits as signals are used to signal start and end of frames.

- In 4B/5B line code 4 data bits are mapped to 5 signal bits to ensure sufficient bit transitions.
- We can use some reserved signals to indicate the start and end of frames.
- It is easy to find the start and end of frames and there is no need to stuff the data.

How to make sure all frames are eventually delivered to the network layer at the destination and in the proper order?

- Positive or Negative acknowledgements.
- How to handle hardware errors or transmission losses?

- Timers
- How to handle duplicate frames?

- Sequence numbers.

- **Feedback based flow control:** the receiver sends back information to the sender giving it permission to send more data, or at least telling the sender how the receiver is doing.
- **Rate based flow control:** the protocol has a built-in mechanism that limits the rate at which senders may transmit data, without using feedback from the receiver.

# Error Detection and Error Correction

Depends on channel characteristics what mechanism is employed.

- Two basic strategies for dealing with errors. Both add redundant information to the data that is sent.
- Error-correcting codes (Forward Error Correction) and Error-detecting codes .



# Error Detection and Error Correction

- Neither correction nor detection can handle all possible errors since the redundant bits that offer protection are as likely to be received in error as the data bits.
- To avoid undetected errors the code must be **strong enough to handle the expected errors**.

# Error types

- Extreme values of thermal noise that overwhelm the signal briefly and occasionally, giving rise to isolated single-bit errors.
- Errors that tend to come in bursts. Deep fade on a wireless channel or transient electrical interference on a wired channel.
- Erasure channel, analog signal that was far from the expected value for a 0 or 1 and declared the bit to be lost.

# Error Detection and Error Correction

- On channels that are highly reliable (fiber) it is cheaper to use an error-detecting code and just retransmit the occasional block found to be faulty.
- On channels that make many errors (wireless), it is better to add redundancy to each block so that the receiver is able to figure out what the originally transmitted block was.

# Error Detection and Error Correction contd.

- Error-correcting codes are also seen in the physical layer, particularly for noisy channels, and in higher layers, particularly for real-time media and content distribution.
- Error-detecting codes are commonly used in link, network, and transport layers.

# Error Correcting Codes

- Hamming codes.
- Reed-Solomon codes.
- Low-Density Parity Check codes.

All of these codes add redundancy to the information that is sent. A frame consists of **m** data (i.e., message) bits and **r** redundant (i.e. check) bits.

# Block, Systematic, Linear Codes

- In a block code, the  $r$  check bits are computed solely as a function of the  $m$  data bits with which they are associated.
- In a systematic code, the  $m$  data bits are sent directly, along with the check bits, rather than being encoded themselves before they are sent.
- In a linear code, the  $r$  check bits are computed as a linear function of the  $m$  data bits.

These are three independent properties. A code may have any combination of them. The Hamming code we study next is all three at once.

# Hamming Codes: Some Basics

- Let the total length of a block be  $n$  (i.e.,  $n = m + r$ ).
- An  $n$ -bit unit containing data and check bits is referred to as an  $n$ -bit codeword.
- The code rate is the fraction of the codeword that carries information that is not redundant, or  $m/n$ .

# Hamming Distance

- Given any two codewords, the number of bit positions in which they differ is called the **Hamming distance** between them.
- Its significance: if two codewords are a Hamming distance  **$d$**  apart, it will require  **$d$**  single-bit errors to convert one into the other.
- The **Hamming distance of the complete code** is the smallest Hamming distance between any two of its legal codewords.



# Hamming Codes: Sparseness

- All  $2^m$  possible data messages are legal, but due to the way the check bits are computed, not all of the  $2^n$  possible codewords are used.
- When there are  $r$  check bits, only the fraction  $2^m/2^n$  or  $1/2^r$  of the possible codewords will be legal.
- It is the sparseness with which the message is embedded in the space of codewords that allows the receiver to detect and correct errors.

# Distance and Error Control

The error-detecting and error-correcting properties of a block code depend on its Hamming distance.

- To reliably **detect**  $d$  errors, you need a distance  $d + 1$  code: no  $d$  single-bit errors can turn one valid codeword into another.
- To **correct**  $d$  errors, you need a distance  $2d + 1$  code: even with  $d$  changes, the original codeword remains the closest legal one.

# Distance and Error Control: Example

Consider a code with only four valid codewords:

0000000000, 0000011111, 1111100000, 1111111111

- This code has a distance of 5, so it can correct double errors or detect quadruple errors.
- If 0000000111 arrives and we expect at most double errors, the receiver decodes it as 0000011111.
- If a triple error changes 0000000000 into 0000000111, it is corrected wrongly.
- Alternatively, if we only detect, all such errors are caught.

# How Many Check Bits? (Single-Error Correction)

- Each of the  $2^m$  legal messages has  $n$  illegal codewords at distance 1 (invert each of the  $n$  bits in turn).
- So each legal message needs  $n + 1$  bit patterns dedicated to it: itself plus its  $n$  neighbours.
- Since the total number of patterns is  $2^n$ :  $(n + 1) 2^m \leq 2^n$ .
- Using  $n = m + r$ , this becomes:  $(m + r + 1) \leq 2^r$

For  $m = 7$ :  $r = 3$  gives  $11 \leq 8$  (fails),  $r = 4$  gives  $12 \leq 16$  (holds). So we use  $r = 4$  check bits, as in the construction next.

# Hamming Codes: Bit Positions and Coverage

- Number the codeword positions 1 to  $n$ . Positions that are powers of 2 (1, 2, 4, 8, ...) hold **check bits**; the rest (3, 5, 6, 7, 9, ...) hold the  **$m$**  data bits.
- To find which message bits contribute to a given check bit  $p_i$  (at position  $i$ , a power of 2), include every position  $k$  whose binary expansion contains  $i$ . Equivalently, this is every  $k$  for which  $i$  appears in  $k$  written as a sum of powers of 2.
- Example:  $11 = 1 + 2 + 8$ , so position 11 is checked by  $p_1, p_2, p_8$ ;  
 $29 = 1 + 4 + 8 + 16$ .

Coverage groups ( $m = 7$ ,  $n = 11$ ):

- $p_1$ : 3, 5, 7, 9, 11       $p_2$ : 3, 6, 7, 10, 11
- $p_4$ : 5, 6, 7       $p_8$ : 9, 10, 11

# Hamming Codes: Computing the Check Bits

- Each check bit forces the modulo-2 sum (parity) of the bits in its group, including itself, to be even.
- So each check bit is the XOR of the data bits in its coverage group.
- A data bit may be included in several check-bit computations, one for each power of 2 in its position number.

This makes the Hamming code **linear** (check bits are XORs of data bits) and **systematic** (data bits appear directly in the codeword).

# Hamming Codes: Worked Example

Message A = 1000001 ( $m = 7$ ). Data bits fill positions 3,5,6,7,9,10,11; computing each check bit as even parity gives the sent codeword:

0 0 1 0 0 0 0 1 0 0 1

- A 1-bit error flips position 5: received 0 0 1 0 1 0 0 1 0 0 1.
- Receiver recomputes each group's parity, forming the **syndrome**  $p_8p_4p_2p_1$ .
- Here  $p_4$  and  $p_1$  fail: syndrome = 0101 = 5. The syndrome *is* the error position.
- Flip bit 5, strip the check bits, recover 1000001 = A.

# Reed Solomon Codes

Reed-Solomon codes are also block-based error correcting codes with a wide range of applications in digital communications and storage. RS codes are used to correct errors in many systems including:

- Storage devices(including tape, Compact Disk, DVD, barcodes, etc)
- Wireless or mobile communications
- Satellite communications
- Digital television / DVB
- High-speed modems such as ADSL etc.



# Reed Solomon Codes

- RS codes work on **symbols** (typically bytes), not individual bits. An  $RS(n, k)$  code adds  $n - k$  check symbols to  $k$  data symbols.
- It can correct up to  $(n - k)/2$  symbol errors, wherever they fall in the block.
- Because a burst of bit errors usually damages only a few consecutive *symbols*, RS corrects burst errors that would overwhelm a bit-oriented code like Hamming.
- RS codes are based on polynomial arithmetic over finite (Galois) fields, typically  $GF(256)$  for byte-sized symbols.

# Error-Detecting Codes

3 different error-detecting codes, Parity, Checksums, Cyclic Redundancy Checks (CRCs).

# Error-Detecting Codes

A single parity bit is appended to the data.

- The parity bit is chosen so that the number of 1 bits in the codeword is even (or odd).
- For example, when 1011010 is sent in even parity, a bit is added to the end to make it 10110100.
- A code with a single parity bit has a distance of 2, since any single-bit error produces a codeword with the wrong parity. This means that it can detect single-bit errors.

# Error-Detecting Codes

Consider a channel on which errors are isolated and the error rate is  $10^{-6}$  per bit (Typical LAN links provide bit error rates of  $10^{-10}$ ).

- Let the block size be 1000 bits.
- To provide **error correction** for **1000-bit blocks**, what is number of check bits required?
- How many check bits are required for megabit of data?
- To merely detect a block with a single 1-bit error, one parity bit per block will suffice.
- Once every 1000 blocks, a block will be found to be in error and an extra block (1001 bits) will have to be transmitted to repair the error.

# Error-Detecting Codes

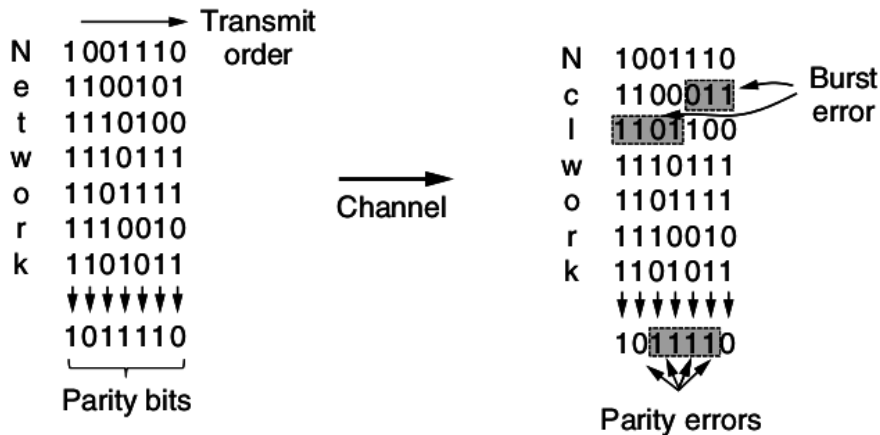
- A single parity bit can only reliably detect a single-bit error in the block.
- If the block is badly garbled by a long burst error, the probability that the error will be detected is only 0.5, which is hardly acceptable.

# Error-Detecting Codes: Burst Errors

To better protect against burst errors, we can compute the parity bits over the data in a different order than the order in which the data bits are transmitted.

- **Interleaving** will compute a parity bit for each of the  $n$  columns and send all the data bits as  $k$  rows, sending the rows from top to bottom and the bits in each row from left to right in the usual manner.
- At the last row, we send the  $n$  parity bits.
- Interleaving is a general technique to convert a code that detects (or corrects) isolated errors into a code that detects (or corrects) burst errors.

# Error-Detecting Codes



# Error-Detecting Codes:Checksum

Checksum is a group of check bits associated with a message, regardless of how those bits are calculated.

- **Ex:** A group of parity bits
- **Ex:** Running sum of the data bits of the message. The checksum is usually placed at the end of the message, as the complement of the sum function. This way, errors may be detected by summing the entire received codeword, both data bits and checksum. If the result comes out to be zero, no error has been detected.



IP header checksum is calculated over IP header only.

- IP header checksum is: 16 bit one's complement of the one's complement sum of all 16 bit words in the header.
- On verification, the one's complement sum of all 16 bit words in the header, including the checksum field, is 0xFFFF. Its complement is zero if there is no corruption.

For example:

4500 0073 0000 4000 4011 **b861** c0a8 0001 c0a8 00c7

To calculate the checksum,  $4500 + 0073 + 0000 + 4000 + 4011 + c0a8 + 0001 + c0a8 + 00c7 = 2479C = 2 + 479C = 479E$ .

One's complement of this result: B861.

# Error-Detecting Codes

CRC (Cyclic Redundancy Check), also known as a polynomial code.

- Polynomial codes are based upon treating bit strings as representations of polynomials with coefficients of 0 and 1 only.
- A k-bit frame is regarded as the coefficient list for a polynomial with k terms, ranging from  $x^{k-1}$  to  $x^0$ .
- Such a polynomial is said to be of degree k-1.
- The high-order (leftmost) bit is the coefficient of  $x^{k-1}$ , the next bit is the coefficient of  $x^{k-2}$ , and so on.
- Write polynomial for 110001?

# Error-Detecting Codes

Polynomial arithmetic is done modulo 2. It does not have carries for addition or borrows for subtraction. Both addition and subtraction are identical to exclusive OR.

- Long division is carried out in exactly the same way as it is in binary except that the subtraction is again done modulo 2.
- When the polynomial code method is employed, the sender and receiver must agree upon a generator polynomial,  $G(x)$ , in advance.
- Both the high and low-order bits of the generator must be 1.
- To compute the CRC for some frame with  $m$  bits corresponding to the polynomial  $M(x)$ , the frame must be longer than the generator polynomial.

# Error-Detecting Codes

- The idea is to append a CRC to the end of the frame in such a way that the polynomial represented by the checksummed frame is divisible by  $G(x)$ .
- When the receiver gets the checksummed frame, it tries dividing it by  $G(x)$ . If there is a remainder, there has been a transmission error.

# Algorithm for computing CRC

1. Let  $r$  be the degree of  $G(x)$ . Append  $r$  zero bits to the low-order end of the frame so it now contains  $m + r$  bits and corresponds to the polynomial  $x^r M(x)$ .
2. Divide the bit string corresponding to  $G(x)$  into the bit string corresponding to  $x^r M(x)$ , using modulo 2 division.
3. Subtract the remainder (which is always  $r$  or fewer bits) from the bit string corresponding to  $x^r M(x)$  using modulo 2 subtraction. The result is the checksummed frame to be transmitted. Call its polynomial  $T(x)$ .

## Algorithm for computing CRC

Frame: 1 1 0 1 0 1 1 1 1 1

Generator: 1 0 0 1 1

Diagram illustrating the long division of 10011 by 110110001110000. The quotient is 10011 and the remainder is 10. The diagram shows the step-by-step subtraction of the divisor from the dividend, with the quotient digits being thrown away and the remainder being 10.

```

      1 1 0 0 0 0 1 1 1 0 0 0 0
      1 0 0 1 1
      -----
      1 0 0 1 1
      1 0 0 1 1
      -----
      0 0 0 0 1
      0 0 0 0 0
      -----
      0 0 0 1 1
      0 0 0 0 0
      -----
      0 0 1 1 1
      0 0 0 0 0
      -----
      0 1 1 1 1
      0 0 0 0 0
      -----
      1 1 1 1 0
      1 0 0 1 1
      -----
      1 1 0 1 0
      1 0 0 1 1
      -----
      1 0 0 1 0
      1 0 0 1 1
      -----
      0 0 0 1 0
      0 0 0 0 0
      -----
      1 0
  
```

← Quotient (thrown away)  
 ← Frame with four zeros appended  
 ← Remainder

Transmitted frame: 1 1 0 1 0 1 1 1 1 0 0 1 0 ← Frame with four zeros appended minus remainder

# Error-Detecting Codes: CRC Properties

Common standardized generator polynomials:

- CRC-16:  $G(x) = x^{16} + x^{15} + x^2 + 1$
- CRC-CCITT:  $G(x) = x^{16} + x^{12} + x^5 + 1$
- CRC-32 (Ethernet):  $G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

A generator with  $r$  check bits (degree  $r$ ) detects:

- All single-bit errors, provided  $G(x)$  has at least two terms.
- All double-bit errors, provided  $G(x)$  has a factor with at least three terms.
- Any odd number of errors, provided  $G(x)$  contains the factor  $(x + 1)$ .
- All burst errors of length  $\leq r$ .
- A burst of length  $r + 1$  with probability  $1 - 2^{-(r-1)}$ , and any longer burst with probability  $1 - 2^{-r}$ .

# The End